Design Patterns in a Django Food-Ordering Application

23CSE455 Case Study — Team A1 (CSE A)

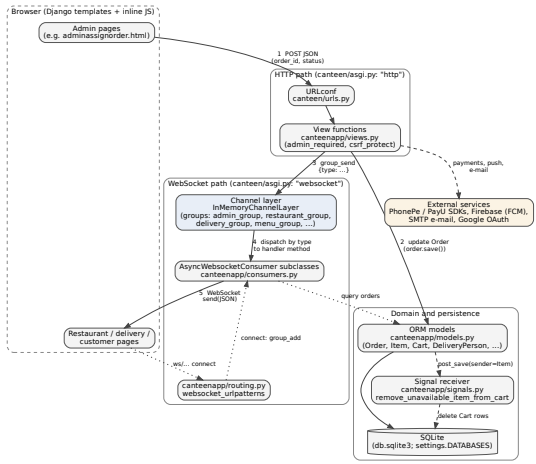
Aadil Nizar Adil Najeem Athul Yesudas Hima C M Krishnapriya Muhammed Faais A

2026-10-11

Application: HarshavardhanVemali/campus-food-ordering, commit 83998629. Full report: `A1_Report.pdf`.

Application and architecture

- One Django app (`canteenapp`); logic mainly in function-based views (5,363 lines).
- HTTP and WebSockets through ASGI; Channels `InMemoryChannelLayer`; SQLite.
- Order-status flow: view saves order → `group_send` → consumers push JSON to browsers.



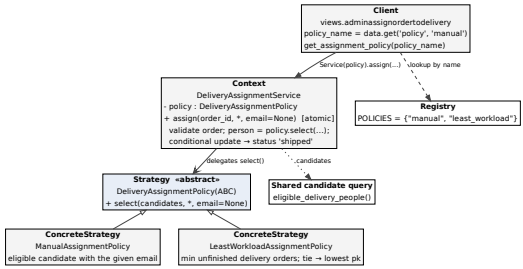
Part 1: patterns at the baseline

Instance	Pattern	Mechanism
Item Unavailability Cart Cleanup	Observer	`post_save` signal clears carts
Order Status Broadcast	Observer (pub–sub)	Channels groups, 3 consumers

- Totals: 1 type, 2 instances, 3 concrete classes, 13 participant symbols.
- 12 candidates rejected (Strategy, State, Singleton, Facade, Adapter, . . .), 2 uncertain.
- The delivery-assignment event has no live subscriber: its consumer is inside a string literal.

Part 2: CH01 — delivery-assignment policies (Strategy)

- `DeliveryAssignmentPolicy` with `Manual` and `LeastWorkload` policies; `DeliveryAssignmentService` as context.
- Least workload: fewest unfinished delivery orders, ties by lowest user ID.
- 39 Django tests and 9 UI script tests pass; no migration needed.
- Counts after CH01: 2 types, 3 instances, 5 concrete classes.
- CH02 and CH03 were not implemented.



Part 3: cross-language comparison

Language	Obs. LOC	Obs. CC	Str. LOC	Str. CC	Median (ms)*
Java	45	2	60	5	70.7
Python	31	2	39	4	18.9
JavaScript	44	1	49	3	34.7
C++	50	2	62	5	1.9

- All four pass Observer 3/3, Strategy 2/2 and the whole-output check.
- *Whole-process time of the combined program (both patterns); mostly runtime start-up.
- Java/C++ enforce the abstraction at compile time; Python/JavaScript rely on duck typing.

LLM use and findings

- Historical analyses survive only as pasted outputs; the report-production prompts (Claude Opus 5.5, 2026-10-11) are saved in `data/llm/`.
- Observer appears twice, both times through framework mechanisms rather than hand-written subjects.
- CH01 adds a tested Strategy that keeps validation, persistence and notification in one context.
- Baseline dependency pins conflict; the tests verify feature behaviour, not deployment readiness.